

Natural Language Processing – CS6120

Assignment 3

Total Mark: 100%+5% bonus

Word Representation

Introduction

The goal of this assignment is to create deeper insights into the world of word representation and word embeddings. One of the challenging topics in word embedding space is how to appropriately evaluate the quality of word embeddings. As you may recall from the class discussion, word embeddings are often known as a representation of a word's semantics or meaning.

PART I (50%)

Understanding Frequency-Based and Distributed Representations

1.1 Data Preparation

Download the provided data corpus from this link and store it in proper data format in Python:
<https://www.kaggle.com/datasets/manann/quotes-500k>

The data consists of:

- **Column 1:** `quote` (Text spoken by the author)
 - **Column 2:** `author` (Speaker's name)
 - **Column 3:** `category` (the category tags for the quote)
-

1.2.1: Identify unique author names. Combine all quotes by each author into a single document.

Hint: Group by author name and preprocess text (lowercasing, removing punctuation, and stopwords).

1.2.2: Find the **two most frequent words (minimum 4 letters)** for three authors of your choice. Calculate their TF-IDF representations and compare the values for each document.

- **Deliverable:** Implement TF-IDF from scratch.

1.2.3: Train a Word2Vec skip-gram model on the corpus using **Gensim**. Experiment with the following two configurations:

- [Context window size: 3, Vector dimensions: 50, Negative samples: 3]
- [Context window size: 7, Vector dimensions: 150, Negative samples: 10]

Compare the models on tasks like:

- `w2v_model.wv.most_similar(positive=["imagination"])`
- `w2v_model.wv.similarity("fear", "knowledge")`

1.2.4: Create a **TSNE visualization** of embeddings for key words such as "imagination", "fear", "understanding", "discovery", and "science" for different configurations of the Word2Vec model.

Hint: Use `tsnecatterplot()` from the Gensim tutorial.

1.2.5: Compare embeddings for the above-mentioned words. Discuss how context and dimensionality impact the relationships in the embeddings.

PART II (50%)

Introduction

This part of the assignment is meant to exercise how one might evaluate word meaning representations, both intrinsically and extrinsically on a text classification task.

Analogy Task

Similar to language models, embeddings are trained directly from a large collection of natural language text without being tied to a specific NLP application or subtask. How can we measure the quality of learned embeddings? Some of the commonly accepted evaluation methods are based on word similarity tasks, word analogies (e.g., “man is to woman as king is to queen”), and semantic compatibilities between verbs and their semantic arguments (also known as “selectional preferences”; see [Schnabel et al., 2015](#)). In this assignment, you will explore an analogy task.

The analogy prediction task is defined as follows. Given a pair of words $\langle a, b \rangle$ and a third word c , choose a fourth word d so that the analogy “ a is to b as c is to d ” holds. In other words, the relationship between c and d should be as close as possible to that between a and b .

Note: the system need NOT characterize this relationship or give it a name! The relationship is taken to be implicit in the pair $\langle a, b \rangle$.

[Mikolov et al., 2013](#) proposed that simple algebraic operations could be applied to embeddings to find an analogy prediction. Let v_a be the vector for a , v_b the vector for b , and so on. For the d such that the analogy holds, we expect

$$v_b - v_a \approx v_d - v_c \quad (1)$$

We therefore seek:

$$d = \arg \max_{d \in V \setminus \{a, b, c\}} \cos(v_d, v_b - v_a + v_c) \quad (2)$$

Note that the given words a , b , and c are excluded from consideration. The cosine similarity is often used as a similarity function between nonzero vectors.

Analogy Dataset

You will be given a subset of Mikolov's analogy dataset, which includes four semantic relations and four syntactic relations. In the test files, each line represents one analogy question, in the form of four words $\langle a, b, c, d \rangle$. For example:

Bangkok Thailand Cairo Egypt

A question is counted as correctly answered only if the predicted word is the same as the given word. For example, given the first three words "Bangkok Thailand Cairo", the task is to predict "Egypt". The full set of analogy questions can be found in the file `word-test.v1.txt` in the assignment data file¹. The groups of relations are delimited by lines starting with a colon (:) and you should ONLY work with the following eight groups:

- capital-world
- currency
- city-in-state
- family
- gram1-adjective-to-adverb

¹ This was downloaded from: <https://github.com/viniciusarruda/word2vec/blob/main/dataset/word-test.v1.txt>

- gram2-opposite
- gram3-comparative
- gram6-nationality-adjective

Word Embeddings

“Pretrained” word embeddings are word embeddings that are already constructed in advance of your NLP project (whether your project is a neural language model, a text classifier, or a class assignment). The advantage of pretraining is that it simplifies learning your model, because the embedding parameters are fixed in advance. The disadvantage is that, if your embeddings happen to be bad for your task, you’re stuck with them.

For this assignment, you are free to use any pretrained word embeddings you can find, as long as you cite the reference. Here are some popular choices:

- Word2vec [Mikolov et al., 2013]: <https://code.google.com/archive/p/word2vec/>
- GloVe [Pennington et al., 2014]: <http://nlp.stanford.edu/projects/glove/>
- Dependency-based embeddings [Levy and Goldberg, 2014]: <https://levyomer.wordpress.com/2014/04/25/dependency-based-word-embeddings/>

Note that embeddings vary not only in the method of construction from data, but also in dimensionality, amount and type of text used, and more. When you report on your choices, be precise!

2.1 [25%] Run the analogy test. Pick any two sets of pretrained embeddings, implement the analogy prediction method described in Equation 2, and compare their accuracies on the eight analogy tasks listed above. Make sure to mention the details of your selection in writing.

2.2 [25%] One known problem with word embeddings is that antonyms (words with meanings considered to be opposites) often have similar embeddings. You can verify this by searching for the top 10 most similar words to a few verbs like *increase* or *enter* that have clear antonyms (e.g., *decrease* and *exit*, respectively) using the cosine similarity. Discuss why embeddings might have this tendency.

2.3 [OPTIONAL for 5% bonus points] Design two new types of analogy tests that are NOT part of Mikolov’s analogy dataset. You will create your own test questions (3 questions for each type, so in total 6 new questions). Report how well the two sets of embeddings perform on your test questions. You’re encouraged to be adversarial so that the embeddings might get an accuracy of zero! Discuss any interesting observations you have made in the process.

Final Deliverables

- A modularized Python script or Jupyter notebook for EACH part of the assignment containing all the functions and their documentation. The code should include appropriate outputs for all intermediate steps. The code should be self-explanatory and easy to execute and evaluate performance.
- A one/two-pager report (pdf) explaining your methodology, results and comparisons.